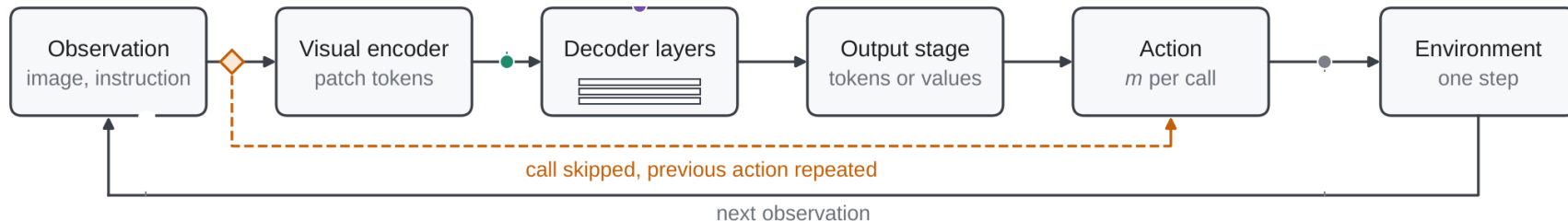


Motivation Efficient VLA inference without retraining



One control step of a frozen VLA policy: the vision encoder and the language backbone run at every step.

[1] VLA inference is slow. Hundreds of milliseconds per step, which limits how often the policy can observe, decide, and correct.

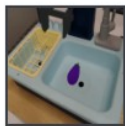
[2] Training-free tricks are reported one at a time. Foveation, action repeat, depth pruning, guarded reuse, temporal fusion: each on one backbone, one benchmark, one implementation.

Do the gains transfer across VLA models and environments, **or are they configuration-dependent?**

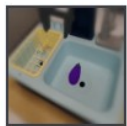
Trick 1 Visual foveation

What to see

Foveation

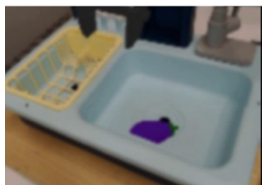


input

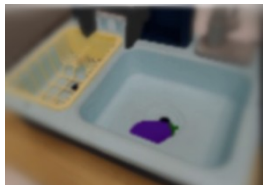


foveated

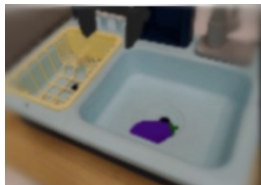
The image is blurred outside a central disc, the token count unchanged.



original



keep 20%



keep 50%

- Sharp central disc, progressively blurred periphery
- Keep ratio 20% or 50%; token count unchanged
- Tests whether high-frequency detail is needed only near the objects and the gripper



CLIP: original vs foveation

same episode, same initial state

Rollout: original (left) and foveated observation (right), success or failure badge at the end.

Trick 2 Action repeat

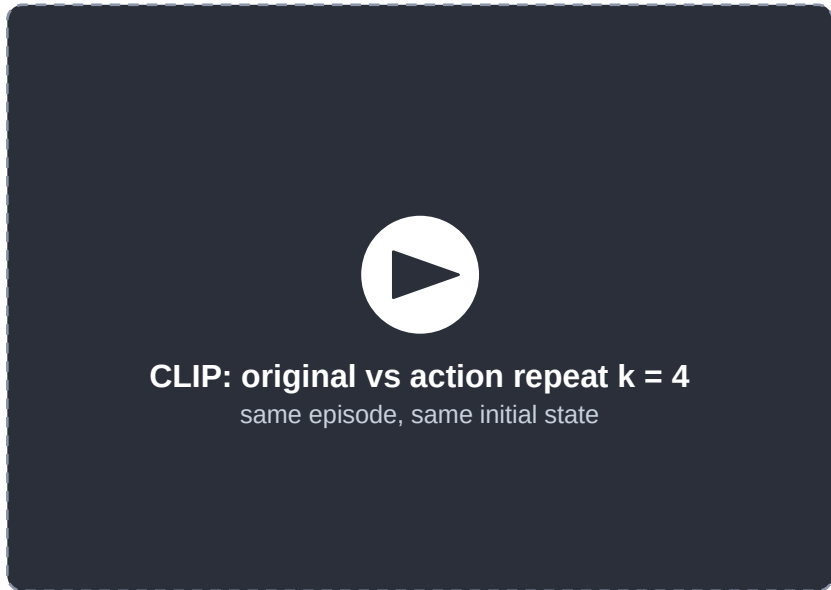
When to act



Policy calls over 12 control steps



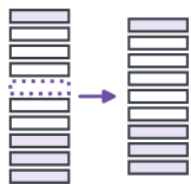
- Hold each predicted action for $k = 2$ or 4 steps, one call per k steps
- Cuts policy calls by about k , but lengthens the open-loop interval
- Fixed reduction, no state check: a speed-oriented baseline



Rollout: watch contact and placement, where a held action can miss a needed correction.

Trick 3 Depth pruning

Depth pruning



The layers lowest in Block Influence are removed, none adjacent, the first layers and the last kept.

- Block Influence: how little a decoder block changes its hidden states on calibration data
- Remove the 1, 2, or 4 lowest-influence blocks; none adjacent, first and last kept
- Selected once, fixed before evaluation; reduces prefill and decoding compute

How much to compute



CLIP: original vs depth pruning

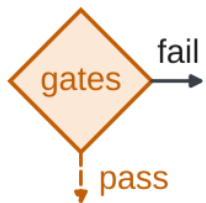
same episode, same initial state

Rollout: CogACT on WidowX, where pruning removes decoder layers without breaking the closed loop.

Trick 4 Guarded action reuse

When to act

Guarded reuse



While the image and action gates pass, the call is skipped and the previous action repeated, up to a cap.

guarded



hollow = call skipped, green = every gate passed

- Reuse the previous action only when all gates pass: small global and local image change, recent actions agree, translation not near zero, gripper unchanged, reuse count below a cap
- Presets: strict, moderate, aggressive



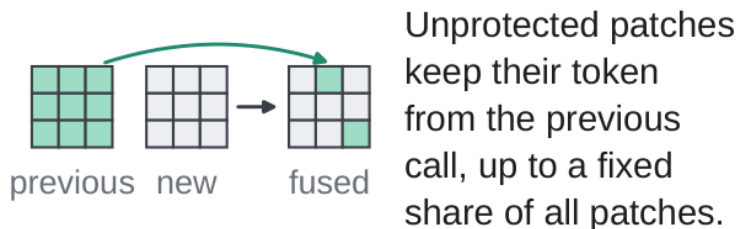
CLIP: original vs guarded reuse

same episode, same initial state

Rollout: overlay the gate indicators and a "calls skipped" counter on the right pane.

Trick 5 Temporal fusion

Temporal fusion



- Reuse projected visual tokens of temporally stable patches from the previous call
- Protect patches with motion, high entropy, high language attention, or interaction regions; recompute those
- The same reuse mask is shared with the generation cache
- Presets: motion-entropy, task-aware, conservative-adaptive

How much to compute



CLIP: original vs temporal fusion

same episode, same initial state

Rollout: optionally tint reused patches on the right pane to show what is carried over between frames.

Results: matched evaluation protocol

Backbone	WidowX	Fractal	LIBERO
CogACT (7.6B)	•	•	
OpenVLA (7B)	•	•	•
SpatialVLA (4B)	•	•	
CronusVLA (0.5B)	•	•	
UniVLA (8.5B)	•		•
MiniVLA (1B)	•		
SmolVLA (0.45B)			•

SimplerEnv WidowX: 4 tasks × 50 episodes. Google Robot/Fractal: 5 tasks × 50. LIBERO: Long, Goal, Object, Spatial, each suite a separate pair (3 × 4 = 12), giving 22 pairs in total.

22

backbone ×
environment pairs

13 + 1

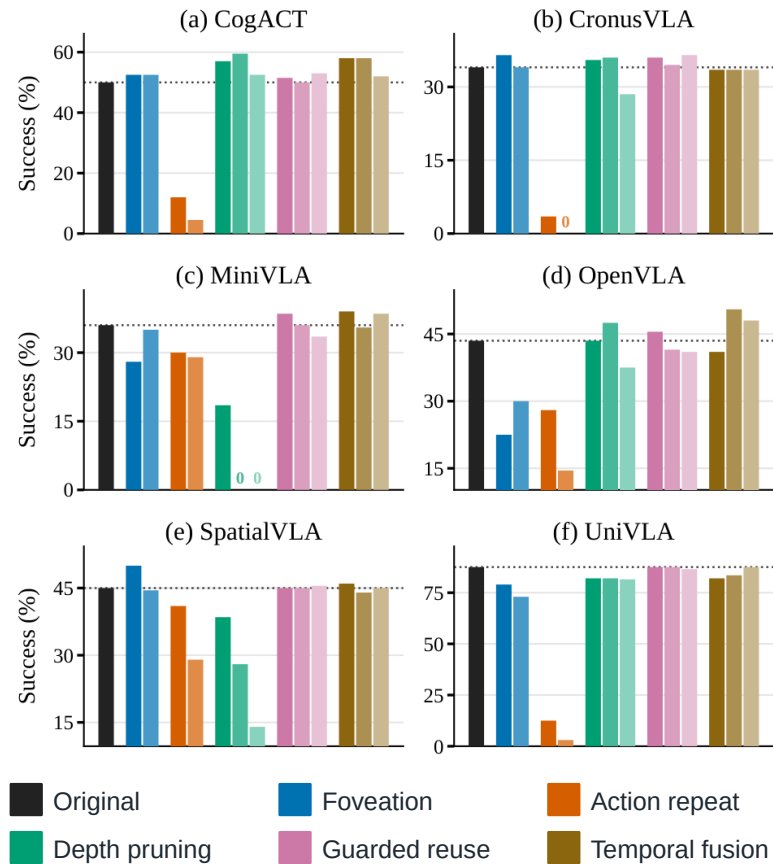
trick settings +
original, per pair

286

trick settings, all
paired with the
original

- Official checkpoints, no retraining
- Same task instances and initial states as the original
- Latency = wall-clock per environment step
- Exact two-sided McNemar test on paired episodes
- 4 × RTX 5090; tables report the best setting per trick

Which tricks preserve success?



Q1 Success

Guarded reuse and temporal fusion are the safest for preserving success.

Selected backbones also gain:

Foveation, WidowX: CogACT +2.5 · SpatialVLA +5.0 · CronusVLA +2.5

Depth pruning, WidowX: CogACT +9.5

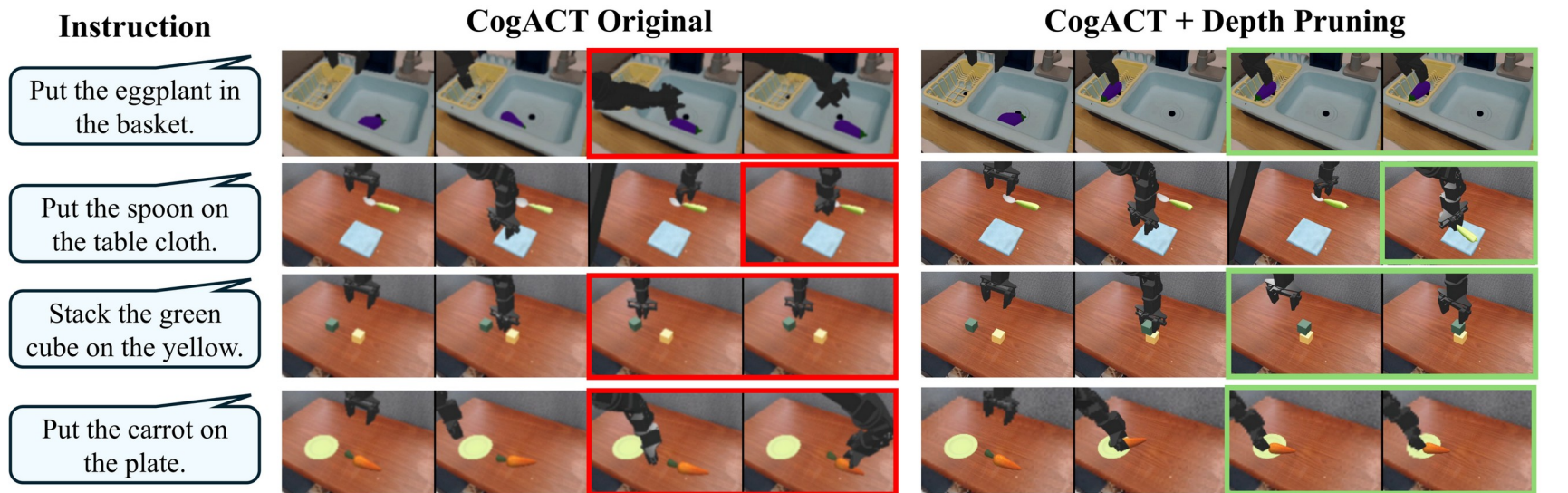
Backbone-dependent:

Foveation, LIBERO Goal: UniVLA +7.0 vs SmoIVLA -13.0

Action repeat: mainly a speed baseline

Fig. 1: success per configuration, six WidowX backbones (14 bars each, darker to lighter within a trick). Dotted line = original policy. Deltas from Tables I and II, best setting per trick.

Depth pruning helps CogACT on WidowX



▶ CLIP: original vs depth pruning (paired episode)

Success 50.0 → 59.5 %

Latency 141.4 → 137.5 ms/step

Red = failure, green = success. Selected episodes, same task instances and initial states. Aggregate values from Table I.

Same trick, same benchmark, different backbone



CLIP: MiniVLA original
succeeds (green border)



CLIP: MiniVLA + pruning
fails (red border)

Depth pruning, WidowX

CogACT +9.5

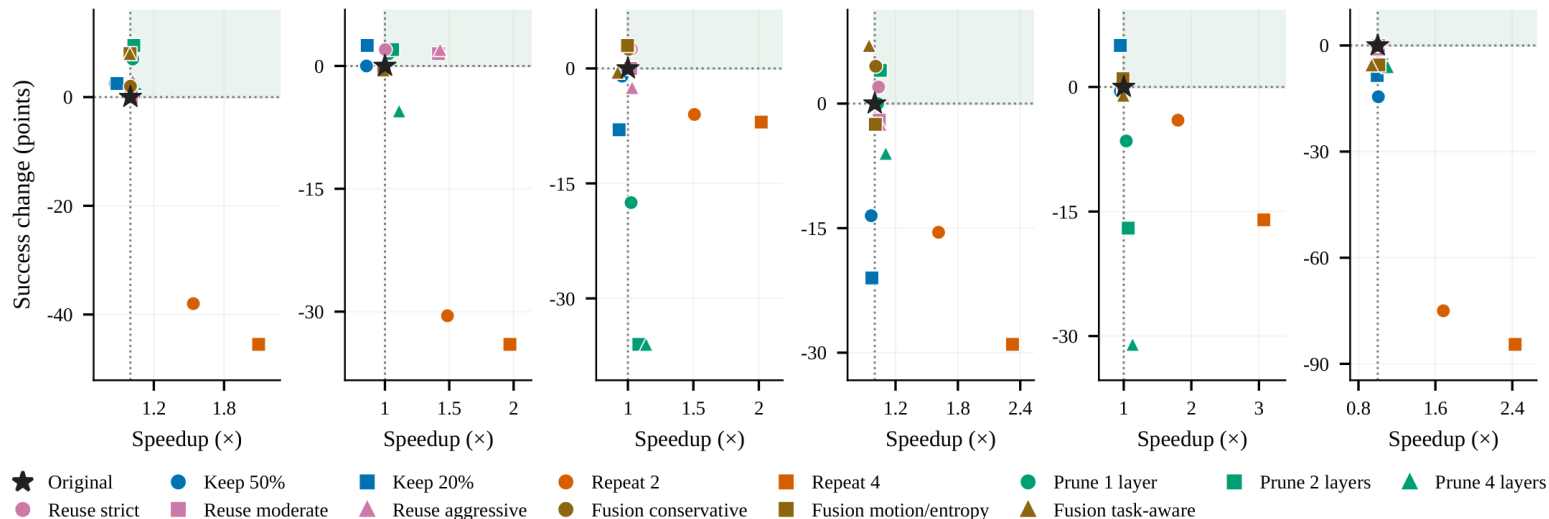
MiniVLA -17.5

SpatialVLA -6.5

Selected episode, same initial state, WidowX

Structural redundancy is not a property of the trick alone. It depends on how each backbone uses its decoder layers during control.

Which tricks are actually faster?



Six WidowX backbones, left to right: CogACT, CronusVLA, MiniVLA, OpenVLA, SpatialVLA, UniVLA. Star = original, shaded = faster and no worse, colour = trick, symbol = setting.

Q2 Speed

Action repeat gives the largest speedup, depth pruning gives decoder-level savings, guarded reuse gives the safest trade-off.

Latency, ms per step	Original → trick
Repeat · SpatialVLA WidowX	423.5 → 235.1
Repeat · OpenVLA LIBERO Long	171.6 → 93.2
Pruning · OpenVLA WidowX	210.2 → 199.4
Reuse · SmoVLA LIBERO Long	289.5 → 268.9

Open loop has a cost, gating keeps it safe



CLIP: Original

policy call every step

policy calls used: ____



CLIP: Action repeat $k = 4$

held actions, episode fails

policy calls used: ____



CLIP: Guarded reuse

calls skipped only when the gates pass

policy calls used: ____

Action repeat, WidowX: CogACT -38.0 · UniVLA -75.0

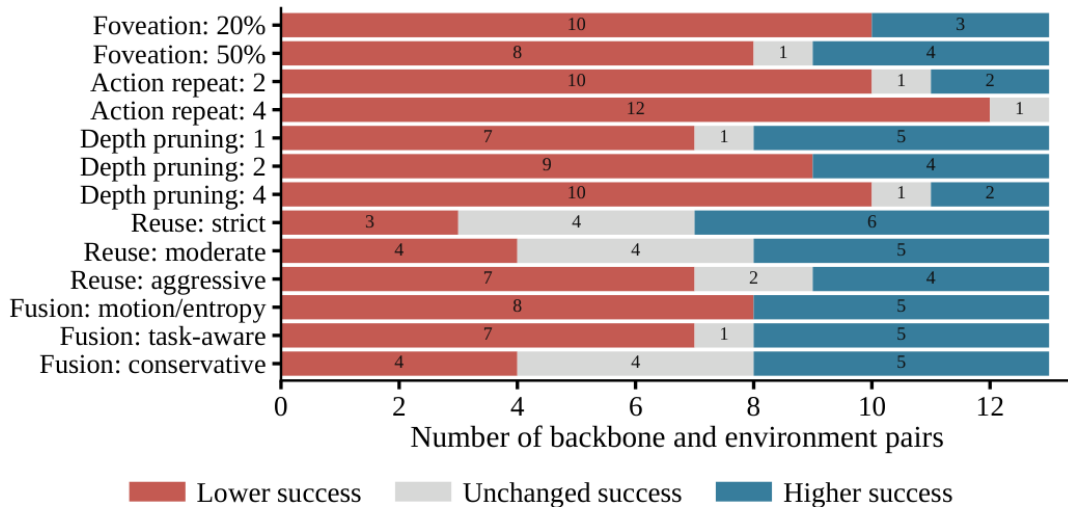
Guarded reuse, SmoVLA LIBERO Long: 289.5 → 268.9 ms

Fractal (milder): CogACT -1.2 · SpatialVLA 0.0

success unchanged at 42.0 %

Same CogACT WidowX episode and initial state in all three panels. Deltas from Tables I and II, best setting per trick.

Does the effect transfer across models?



For each setting: how many of 13 backbone and environment pairs got lower, unchanged, or higher success. LIBERO episodes are pooled over the four suites, so 13 pairs stand for the 22.

Q3 Transfer

Effects are a joint property of the trick, the backbone, and the task distribution.

Depth pruning, WidowX: CogACT +9.5 ·
MiniVLA -17.5

Foveation, LIBERO Goal: UniVLA +7.0 ·
SmoVLA -13.0

Action repeat: harmful on WidowX, milder on
Fractal for some backbones

No trick is plug-and-play: each one needs matched evaluation under the target backbone and environment.

Takeaways

Guarded reuse

Most reliable. Preserves success, modest speedup when the gates open.

Action repeat

Largest speedup, but the risk of long open-loop execution.

Depth pruning

Decoder-level savings, benefit depends on the backbone.

Foveation · Temporal fusion

Test how much the visual representation can change while the policy stays stable.

VLA efficiency is policy- and environment-dependent. Evaluate every trick under the target backbone and environment before applying it.